Behaviour Cloning via Supervised Learning of an Arm Manipulator on a Moving Base

Noah Calati
Computer Science Dept
UCL
noah.calati.26@ucl.ac.uk

Sam Read
Computer Science Dept
UCL
samuel.reed.26@ucl.ac.uk

Finn Gorman
Computer Science Dept
UCL
finn.gorman.26@ucl.ac.uk

***Abstract*—In this paper we explore how to clone the behaviour of an arm manipulator on a moving base. We first collected data in a simulation allowing for a model to interpolate the data and make behavioural decisions to replicate the original motion and ability of the expert. In order to do this we made use of a ridge regressor as a baseline model, an MLP and a GRU in our experiment setup. We evaluated performance via a robustness test, mean squared error and success rate. Both networks reached 95.2% success on held-out test targets (expert 98%), against 0% for Ridge; adding noise to the training commands (DART) and giving the policy memory of the expert's hidden reference mattered far more than lowering offline error.**

I. Introduction

A. Problem Formulation

The robot is comprised of a skid-steer Robotnik base and a 7-DoF Kinova arm, simulated in PyBullet [9] at 1kHz. In each simulation episode the manipulator must be brought within 1 cm of a static target from a random initial base pose and arm configuration. An expert controller solves the task of movement and at every step the policy observes

$$\mathbf{o}_t = \left[\, \mathbf{p}_t^b,\ \psi_t,\ \mathbf{q}_t,\ \dot{\mathbf{q}}_t,\ \mathbf{x}^* \,\right] \in \mathbb{R}^{20}, \tag{1}$$

where $\mathbf{p}_t^b \in \mathbb{R}^2$ and ψ_t are the planar base position and bearing (`base_pos`, `base_bearing`). $\mathbf{q}_t, \dot{\mathbf{q}}_t \in \mathbb{R}^7$ are the measured arm joint positions and velocities (`q_mes`, `qd_mes`). It outputs

$$\hat{\mathbf{a}}_t = \pi_\theta(\mathbf{o}_t) = \left[\, \hat{\boldsymbol{\tau}}_t,\ \hat{\mathbf{u}}_t \,\right] \in \mathbb{R}^9, \tag{2}$$

where $\hat{\boldsymbol{\tau}}_t \in \mathbb{R}^7$ are arm joint torques (`torque_arm`, clipped to $\pm 30\,\mathrm{N\,m}$) and $\hat{\mathbf{u}}_t = [\omega_R, \omega_L]$ are right/left wheel velocities (`wheel_cmds`), applied to the four wheels. The six finger joints are held fixed and are not part of the action. The networks receive features $\phi(\mathbf{o}_t)$ computed from $\mathbf{o}_t$ with the robot's kinematic model, plus the base velocity as one extra sensor.

The expert is not a function of $\mathbf{o}_t$ by itself: $\mathbf{a}_t^* = \pi^E(\mathbf{o}_t, \mathbf{h}_t)$, where the hidden state $\mathbf{h}_t$ holds the navigation phase (align/drive/orient) and an arm joint reference $\mathbf{q}_t^d$ that the expert integrates over time and tracks with a PD law. Because of this, we learn the base and the arm as separate models. Our primary motivation is to improve the closed-loop behaviour when π_θ replaces π^E.

B. Assumptions and Constraints

We assumed flat ground with no obstacles. Between episodes only the target position, the start pose and the start arm configuration change. For robustness tests we additionally vary actuator noise, joint-angle noise and floor friction. The pre-defined expert limits are not changed: if the base has not parked after 25 s, the arm starts reaching from wherever the base stopped, and an episode lasts at most 40 s. In closed loop, the expert's navigation-done flag decides when control passes from the base model to the arm model; this switch is not learned. In simulation a success is considered if the end effector is within 0.01 metres of the target.

C. Contributions

- **Ridge regression baseline:** 0% closed-loop success. An isolation run with the expert driving the base (12.7%) with a per-phase error breakdown.
- **MLP with a reconstructed expert reference:** a second arm MLP predicts the expert's reference velocity, and the policy tracks the offset $\mathbf{e} = \mathbf{q}^d - \mathbf{q}$. Trained with DART data, it reached 95% on test (expert 98%) with 61k parameters.
- **GRU:** learns the expert's hidden state from the demonstrations instead, reaching 95.2% on test and 98.4% on validation.

II. Shared Setup

Fig. 1 gives an overview of the pipeline, from expert data collection to training and closed-loop evaluation.

A. Simulation and Expert Controller

Each episode is assigned a static target position for the end effector. The base controller first drives the robot to a parking pose in front of the target, at a stand-off distance that depends on the target height. It completes this in four phases: align/drive/orient and done, using proportional control on the heading and distance. The forward and turning speeds are converted into right and left wheel speeds for the skid-steer base. While the base is moving the arm holds its starting pose; if the base has not parked within 25,s its marked as done and the arm reaches from where the base is stopped. Once the base has parked, the arm controller asks the end effector to move towards the target at a speed proportional to the error. The

desired end effector velocity is converted into desired joint velocities $\dot{\mathbf{q}}^d$ using the damped least squares inverse of the arm Jacobian [8]. The reference position is then integrated and clipped to within ± 0.15,rad of the measured joint angles, so it cannot wind up when the target is unreachable. A gravity-compensated PD law tracks this reference.

$$\boldsymbol{\tau} = K_p(\mathbf{q}^d - \mathbf{q}) + K_d(\dot{\mathbf{q}}^d - \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}), \tag{3}$$

An episode ends when the end effector is within 1 cm of the target, or after 40 s if the target was unreachable.

The expert controllers were not changed, so all labels came from the original expert. In order to facilitate data generation, parallel simulation, training and inference of the models the robotnik_arm.task file was changed in key places. The main function became run_episode which takes arguments to automate target randomisation, start position, arm offset, noise seed and policy so every episode can be generated automatically. The only change to the expert's behaviour is the base goal is computed from the robot's start position rather than the world origin. Extra parameters where logged for training such as the expert's hidden joint reference $\mathbf{q}^d$ and $\dot{\mathbf{q}}^d$. Gravity torque, shoulder position and joint-limit margins as well as base velocity were also logged. Finally, a learned policy can be passed to run_episode in which case its commands are sent instead of the expert's.

B. Expert Data Generation

To generate expert data, the target position, base position, base orientation and starting arm pose were randomised within set ranges(Table I). The target was sampled in a horizontal distance, bearing and height relative to the robot's start, and each arm joint was given a random offset in 70% of episodes. The main pool of 420 episodes was spread using Latin hypercube sampling [10], which splits the range of each of the 14 sampled variables into 420 equal intervals and places exactly one episode in each, so every variable is covered evenly. Every 1,ms steps were logged and every tenth step was kept for training (split into base navigation and base navigation done pools). A training episode would be finished once the target is reached, when the error stops improving or after 40 seconds. Data quality was assessed by inspecting the stop reason for every episode, checking for NaNs, timing errors and confirming the fingers stay fixed (redundant); crashed episodes produce no data and are left out. Episodes where the expert couldn't reach the target were cut at closest approach. The main data pool was split 70/15/15 into train, validation and test sets by episode rather than by time step, because consecutive samples from the same episodes would leak information between sets. Validation datasets were used to make decisions during development and test sets used to validate performance on completely held out data. Two extra data pools were collected for training only. DART [2] pools were collected with noise ($\sigma = 0.5$) added to the commands sent to the robot, while the clean expert command was logged as the label. Four out-of-range pools of 30 episodes (farther targets, higher targets, start positions outside the training range and larger arm offsets) were used only to test generalisation. Ridge is trained on the main pool only; the MLP and GRU also use the DART pools.

TABLE I
EXPERT DATASET SUMMARY

Property	Value
Episodes (train / val / test)	294 / 63 / 63
Extra training pools (DART)	150 + 500, $\sigma = 0.5$
Control / logging rate (Hz)	1000 / 100
Mean expert time to reach (s)	9.8
Target distance, height (m)	$[0.8, 2.5]$, $[0.15, 1.10]$
Target bearing (rad)	$[-\pi, \pi]$
Initial base x, y (m), yaw	$[-1.5, 1.5]$, $[-\pi, \pi]$
Initial joint offset (rad)	± 0.25 (70% of episodes)
Expert success, val / test (%)	100 / 98

C. Common Input/Output Representation

Our base and arm models are learned as separate models, using the expert's navigation-done flag to decide which one is in control. The expert does two different jobs in sequences; driving the wheels while the arm holds still then parking and moving the arm while the base is still. A single network would have to learn this switch, while separate models can learn a smoother mapping from fewer inputs. Our base model learns on the driving rows from each episode while the arm does on the reaching rows. The base model receives 8 inputs: the target position (3), its distance and bearing and the base's forward, sideways and turning velocity. The velocity is the only sensor beyond $\mathbf{o}_t$. The arm model receives 26: the target position (3), $\mathbf{q}$ and $\dot{\mathbf{q}}$ (14), the distance to the limit of joints 1, 3 and 5, the shoulder-to-target vector (3), which shows whether the target lies beyond 0.85 m and the hand-to-target vector (3) (the error the expert's inverse kinematics acts on). Every input and output is standardised to zero mean and unit variance using statistics from the training set only, so no information from the validation or test episodes is used. Inputs may have different units, and thus, without scaling networks may train less reliably. The outputs are standardised so that each joint counts equally in the loss: joints with larger torques would otherwise dominate this error (NMSE).

D. Common Training Objective

All models are fitted by minimising the mean squared error between their predictions and the expert labels over the training data: $\mathcal{D}$:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{o},\mathbf{a})\in\mathcal{D}} \left\| \pi_\theta(\mathbf{o}) - \mathbf{a} \right\|_2^2, \tag{4}$$

where the labels $\mathbf{a}$ are standardised, so that every output counts equally. Ridge adds an ℓ_2 penalty $\alpha\|\mathbf{w}\|_2^2$, and the GRU weights reaching steps within 5 cm of the target three times as heavily. Hyperparameters for every model (Ridge's α, the early-stopping epoch and the network sizes) were chosen

TABLE II
FINAL HYPERPARAMETERS PER MODEL

Hyperparameter	Ridge	MLP	GRU
Layers × width	–	2 × 128 (3 nets)	1 × 128 (2 nets)[a]
Regularisation	ℓ_2, $\alpha = 0.01$	dropout 0.1	w. decay 10^{-5}[b]
Optimiser / LR	–	Adam / 10^{-3}	AdamW / 10^{-3}
Batch size	–	1024	8 episodes
Epochs / early stop	–	≤100 / 10	40, 80 / best[c]
History length k	1	1 (+ **e**)	full episode
Training data	Main (294 ep.)	Main + DART (294 + 650 ep.)	

[a] GRU layer, plus 128 embedding and 2 × 128 head.
[b] Gradient clipping at 1.0, cosine LR decay.
[c] Base 40, arm 80 epochs; lowest-val-loss epoch kept.

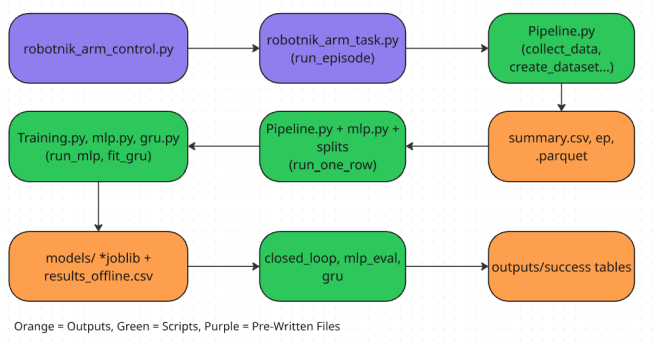


Fig. 1. Overview of the behaviour cloning pipeline.

using the validation episodes only (plus 137 extra training-distribution episodes for the GRU); the test episodes were run once, with the final models. Final values are in Table II.

E. Evaluation Protocol

Every model is evaluated with the same metrics:

- **Accuracy**: offline NMSE on held-out episodes, and closed-loop final error and success rate on the 63 test episodes.
- **Generalisation**: four unseen pools of 30 episodes: farther targets (2.5–3.5 m), higher targets (1.1–1.3 m), start positions outside training (1.5–3.0 m) and larger joint offsets (±0.5 rad).
- **Robustness**: on validation episodes, without retraining: actuator noise ($\sigma = 0.25, 0.5$), floor friction $\mu = 0.4$, and joint-angle noise ($\sigma = 0.01, 0.02$ rad).
- **Efficiency**: parameters, model file size and per-step inference time vs. the 1 ms control period.

The expert's run of the same episodes is the baseline for every result.

III. RIDGE REGRESSION

A. Architecture and Inputs

Ridge regression [4] maps the standardized inputs of Section II-C linearly to the labels, $\hat{\mathbf{a}} = W\phi(\mathbf{o}) + \mathbf{b}$, adding an ℓ_2 penalty $\alpha\|W\|^2$ to Eq. (4). We make use of a linear model as our baseline because the expert's arm torque is linear in the joint errors $\mathbf{q}^d - \mathbf{q}$ and $\dot{\mathbf{q}}^d - \dot{\mathbf{q}}$ plus a gravity term. However, $\mathbf{q}^d$ is not measured, so the Ridge Regression cannot compute these errors directly and must estimate the torque from the joint angles, velocities and the end effector to target distance instead. This shows how well the expert can be copied by a linear model that only sees the robot's current state.

B. Training and Hyperparameter Selection

The Ridge Regression is trained on the main training split (294 episodes), without DART pools [5]. For each model α was chosen from 17 values spaced logarithmically between 10^{-2} and 10^{6} by the lowest validation NMSE, giving $\alpha = 0.01$ for both base and arm.

C. Accuracy and Efficiency

Offline, the arm model reaches a validation NMSE of 0.198, about 5× better than always predicting the mean (1.0), yet this is still too large to succeed in closed loop. The base appears more accurate, but this average is largely made up by straight driving; its error is 5× higher when turning on the spot at the start and 15× higher when turning to face the target. A linear mapping cannot represent the phase switches or the expert's turning law, which saturates at ±1 rad/s once the heading error exceeds about 0.33 rad. In closed loop tests driving both base and arm reached none of the 63 validation episode targets. With the expert driving the base, the Ridge arm reached 12.7% of episodes; in the other 55 the hand stopped improving short of the target (median final error 13.2,cm). Near the target the expert's torques are small and precise, and Ridge's torque error (NMSE 0.198) is too large to make the final few centimetres. The model's efficiency is negligible if it fails to complete the task entirely.

D. Generalisation and Robustness

Because the full Ridge policy reached 0% in-distribution, the out-of-range and robustness tests were not run for it: a policy that already fails cannot be meaningfully made worse.

TABLE III
RIDGE OFFLINE ERROR AND CLOSED-LOOP RESULTS (VALIDATION)

Offline	**NMSE**	
Arm (full torque)	0.198	
Base, all rows	0.048	
turning at start (align)	0.087	
driving (drive)	0.016	
turning at end (orient)	0.246	
Closed loop (63 episodes)	**Succ. (%)**	**Median err.**
Ridge base + Ridge arm	0	170 cm
Expert base + Ridge arm	12.7	13.2 cm
Expert	100	1.0 cm

E. Limitations

A single linear mapping cannot represent the base controller's phase switching and the lack of the expert's hidden joint reference $\mathbf{q}^d$. Both motivate a nonlinear model (Section IV).

IV. Multi-Layer Perceptron

A. Architecture and Inputs

Two MLPs were implemented in PyTorch [7] using the features from Section II-C: a base model (8 inputs $\rightarrow$ 2 wheel commands) and an arm model (26 inputs $\rightarrow$ 7 joint torques). Each hidden layer uses Linear–ReLU–Dropout, and a lightweight wrapper provides a scikit-learn compatible `predict` interface for seamless closed-loop integration.

To emulate the expert controller—which moves an internal reference pose $\mathbf{q}^d$ via velocity $\dot{\mathbf{q}}^d$ governed by a PD law $\boldsymbol{\tau} = K_p(\mathbf{q}^d - \mathbf{q}) + K_d(\dot{\mathbf{q}}^d - \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q})$—a second arm MLP was added. Because $\mathbf{q}^d$ is unmeasured, identical states can have conflicting torque labels, causing a plain MLP to learn their average. The final policy uses two arm MLPs: the first predicts reference velocity $\hat{\dot{\mathbf{q}}}^d$, while the policy tracks the offset $\mathbf{e} = \mathbf{q}^d - \mathbf{q}$:

$$\mathbf{e}_t = \mathrm{clip}\big(\mathbf{e}_{t-1} + \hat{\dot{\mathbf{q}}}^d_t\,\Delta t - (\mathbf{q}_t - \mathbf{q}_{t-1}),\ \pm 0.15\,\mathrm{rad}\big), \quad \mathbf{e}_0 = \mathbf{0}. \tag{5}$$

The second MLP takes the state plus $\mathbf{e}$ (33 inputs) to output torques. Training labels derive from the logged expert reference, providing a structured integrator memory without recurrent architectures.

B. Training and Hyperparameter Selection

All MLPs minimise (4) on standardised labels using Adam [6] (learning rate 10^{-3}, batch size 1024, up to 100 epochs with early stopping). To mitigate compounding error [1], 650 noisy episodes were collected via DART [2] by adding observation/command noise while storing clean expert targets. DAgger was tested but discarded: expert labels based on drifted MLP references caused 12% torque saturation (± 30 N m) and degraded closed-loop performance.

A validation sweep established the final hyperparameters: dropout of 0.1 outperformed 0.2/0.4 on the arm MLP, and 2 hidden layers matched deeper networks while maximising accuracy on all MLPs. A width sweep revealed that 128×2 matches 256×2 closed-loop success while outperforming 64×2 (Table IV). Thus, 128×2 with dropout 0.1 was selected for all networks.

C. Accuracy and Efficiency

Table IV serves as an ablation study of the policy components. The base MLP achieves low offline NMSE (0.033) but succeeds in only 6% of closed-loop episodes due to compounding drift. DART yielded the primary performance jump (up to 83%), while reference tracking ($\mathbf{e}$) added consistent gains (reaching 87% validation, 90% test). Remaining failures are near-misses (0.4–3.6 cm) concentrated on targets below 0.4 m (Fig. 2).

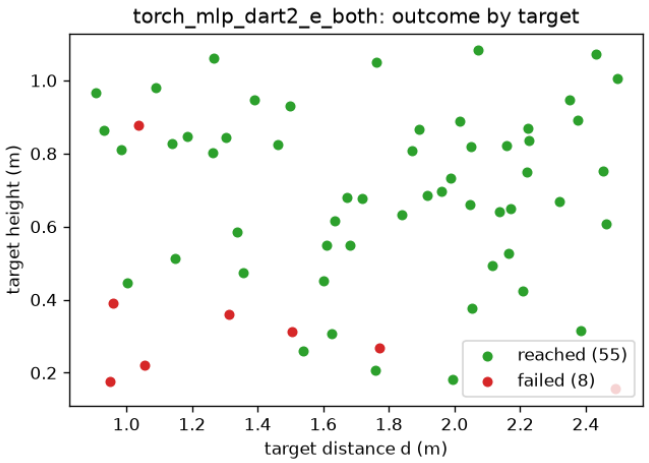


Fig. 2. Closed-loop outcome of the 256×2 MLP by target distance and height on the evaluation episodes.

TABLE IV
MLP Development Steps (Validation, 63 Episodes, Closed Loop)

Variant	Arm NMSE	Params	Succ. (%)
Clean expert data, 256×2	0.033	143k	6
+ DART, 150 episodes	0.029	143k	59
+ DAgger round 1 (discarded)	0.011	143k	30
+ DART, 650 episodes	0.020	143k	83
+ EPolicy, 256×2	0.016	219k	87
EPolicy, 128×2 (final)	0.014	61k	87
EPolicy, 64×2	0.023	18k	79

The final 128×2 model requires $3.6\times$ fewer parameters (61k vs. 219k) and $3.4\times$ less memory (256 vs. 877 kB) than the 256×2 variant, though reach time increased slightly (15.8 s vs. 12.2 s; expert: 9.8 s). On test episodes, the 128×2 model scored 95% vs. the expert's 98%, though this marginal gain over the 256×2 model reflects sample variance across the 63-episode test pool.

D. Generalisation and Robustness

Out-of-distribution evaluations (Table V) show the model interpolates well but struggles to extrapolate, with success dropping to 47% on targets higher than training limits.

Robustness tests on the validation set indicate high resilience: sensor joint-angle noise had negligible effect due to the ± 0.15 rad clipping window preventing error accumulation. Actuator noise up to $\sigma = 0.25$ maintained 87% success, dropping to 79% at $\sigma = 0.5$. Lower floor friction ($\mu = 0.4$) preserved a 92% success rate, confirming base-model robustness.

TABLE V
MLP GENERALISATION AND ROBUSTNESS (SUCCESS %)

Test condition	Expert	MLP
In-distribution targets (test)	98	95
Farther targets	100	100
Higher targets	100	47
Unseen initial base pose	97	90
Unseen initial joint config.	100	80
Robustness (validation, 63 episodes)		
No adversity (val baseline)	100	87
Actuator noise $\sigma = 0.25$	90.5	87
Actuator noise $\sigma = 0.5$	65.1	79
Low floor friction $\sigma = 0.4$	-	92
Sensor noise $\sigma = 0.01$ rad	-	87
Sensor noise $\sigma = 0.02$ rad	-	92

E. Limitations

MLP have no inherent memory function, so without the handwritten error determination the model has no idea where the hidden reference for the arm should be. This is a limitation as it means that the model itself cannot learn this feature, therefore it wouldn't be generalisable to any other controller. A network with memory such as GRU may be better suited to this task as it has inherent memory.

V. GATED RECURRENT UNIT

A. Architecture and Inputs

A Gated Recurrent Unit (GRU) [3] is a recurrent network whose gated hidden state, updated each step, summarises the episode so far. It was chosen as an alternative to the MLP because it can infer the expert behaviour that a single observation doesn't reveal, mainly the base controller's current state and the joint reference the arm controller integrates over time. The MLP reconstructs this reference explicitly; the GRU learns the relevant history from the demonstrations.

The base and arm networks share one architecture: a linear embedding with SiLU activation, a single GRU layer of width 128 (matching the MLP) and a two-layer head combining the hidden state with the current embedded inputs. The base takes 10 inputs and the arm 39, with the arm predicting torques with gravity removed. (Table VI).

B. Training and Hyperparameter Selection

Each expert episode yields one sequence per network (driving rows for the base and reaching rows for the arms), sampled every 10th step, trained in full from a zero hidden state similar to a closed loop. As episode lengths vary, sequences are padded and masked, and episodes of similar length are batched in groups of 8 to keep padding low. The GRU uses the same data as the MLP from Section IV-B, giving 944 training (DART+main) episodes. It was trained on the MSE of Eq. (4) with standardised labels, weighting arm steps within 5 cm of the target $3\times$, as the last few centimetres decide success. AdamW was used with cosine learning-rate decay and gradient clipping. Rather than early stopping, the base and arm were trained for a fixed 40 and 80 epochs, keeping the lowest-validation-loss epoch (38 and 61)(Table VII).

TABLE VI
GRU INPUTS AND OUTPUTS

Network	Inputs (robot frame, standardised)	Dim.
Base	target position; parking-pose position, distance and heading (sin, cos); final heading error (sin, cos)	10
Arm	target position; joint angles $\mathbf{q}$ (raw, sin, cos); joint velocities $\dot{\mathbf{q}}$; hand-to-target error $(x, y, z,$ norm); hand-to-reach-point error $(x, y, z,$ norm)	39
Network	**Outputs**	**Dim.**
Base	right and left wheel speeds, clipped to ± 3.5 rad/s	2
Arm	joint torques $\boldsymbol{\tau} - \boldsymbol{\tau}_g$; $\boldsymbol{\tau}_g$ added back, clipped to ± 30 N m	7

TABLE VII
GRU DEVELOPMENT STEPS (VALIDATION, 63 EPISODES, CLOSED LOOP)

Variant	Arm NMSE	Params	Succ. (%)	Sel. (/200)
Main + DART 150, 40 ep.	0.050	305k	81	–
+ DART, 650 episodes	0.034	305k	92	185
+ DAgger round 1 (discarded)	0.054	305k	81	149
+ $10\times$ near weight (discarded)[a]	0.047	305k	92	188
+ 80 arm epochs (final)	0.027	305k	98	195

[a] Arm steps in the last 3 cm weighted $10\times$ (final uses $3\times$ within 5 cm).
Sel. = successes on val + 137 extra episodes, used to choose the model.

C. Accuracy and Efficiency

Offline, the test NMSE was 0.020 (base) and 0.027 (arm, gravity-removed labels). The validation losses used to pick the epoch (0.023 and 0.020) are not directly comparable for the arm, which weights near-target steps $3\times$ over the latter. In closed loop the GRU succeeded in 60 of 63 test episodes (95.2%), with a final error of 0.92 ± 0.41 cm and a mean reach time of 11.8 s (expert: 9.8 s). All three failures were near-misses on low targets (0.17–0.38 m high), ending 1.1–3.5 cm away, with the hand stalling in a position the expert would push out of. If the experts internal reference pushes the hand out of these stalled positions it suggests the GRU's memory didn't learn the integrator in these stalled states and that these states were absent in training. The base stalled similarly in 6 of 63 episodes, stopping about 16 cm short of its parking pose with wheel commands of $\sim$0.05 rad/s (expert: 1–3 rad/s) until the 25 s timeout counted it as parked. The arm still reached the target in all six, but these episodes are the main cause of the longer reach time (10.0 s without them).

The GRU has around five times more parameters than the MLP, with a 2.4 MB model file compared to 256 kB. At run time it only needs to store 128 values per network. One control step takes around 0.1 ms per step including input computation (the MLP's 102 µs excludes it).

D. Generalisation and Robustness

The GRU succeeded more often than the MLP in out or range tests, but at points of failure it ended far from the

target (9.3 ± 13.7 cm), whereas the MLP's failures were closer suggesting the learned memory does map more smoothly.

TABLE VIII
GRU GENERALISATION AND ROBUSTNESS (SUCCESS %)

Test condition	**Expert**	**GRU**
In-distribution targets (test)	98	95.2
Farther targets	100	100
Higher targets	100	40.0
Unseen initial base pose	97	96.7
Unseen initial joint config.	100	93.3
Robustness (validation, 63 episodes)		
No adversity (val baseline)	100	98.4
Actuator noise $\sigma = 0.25$	90.5	93.7
Actuator noise $\sigma = 0.5$	65.1	84.1
Low floor friction $\mu = 0.4$	100	98.4
Sensor noise $\sigma = 0.01$ rad	100	98.4
Sensor noise $\sigma = 0.02$ rad	100	100

Sensor noise on the joint angles had no negative effect, with 98.4% success at $\sigma = 0.005$ and 0.01 rad and 100% at 0.02 rad, against 98.4% without noise. This means the learned memory did not make the model more sensitive to noise. A possible reason is that the hidden state is built up over many steps, which averages out noisy readings, similar to how the clip on the MLP's reference stops noise from building up. Actuator noise was the hardest condition, with success dropping to 93.7% at $\sigma = 0.25$ and 84.1% at $\sigma = 0.5$. However the expert itself only reached 90.5% and 65.1% under the same noise, so the GRU actually recovered better than the controller it was trained to copy. This could be explained due to DART training data containing recovery examples that the expert never needs to use itself. With a low floor friction of $\mu = 0.4$ success stayed at 98.4%, as the base inputs track the parking pose rather than the wheel motion.

E. Limitations

The GRU does not extrapolate well to targets above the training height and the failures here can be large. Close to the goal it can also undershoot, with the base stalling short of it's parking pose until the 25 s timeout in 6 of the 63 test episodes. More generally, A GRU has to process each episode step-by-step, which makes training slower than a feed-forward network, and it's memory is learned rather than designed, so unlike the MLP's explicit reference it cannot easily be inspected or explained.

VI. COMPARISON AND DISCUSSION

A. Side-by-Side Results

The GRU was the most reliable in closed loop (98.4% val, 95.2% test, 11.8 s to reach) against the MLP's 87% and 95% (15.8 s) and Ridge's 0%, while the expert reached 98% of test targets in 9.8 s. Offline MSE is not comparable across models because the GRU predicts gravity-removed torque while the MLP predicts full torque. In any case, offline error did not predict closed-loop success for the MLP but held partly for the GRU. The GRU's extra reliability comes at five times the MLP's size (305k vs. 61k parameters), though both run well within the 1 ms control period, with generalisation and robustness covered in each model's own section (Tables III, V, VIII).

B. Discussion

The GRU was best on accuracy and robustness, as its memory learns the expert's hidden phase and joint reference directly, while for generalisation both networks did well on unseen start conditions but struggled on higher targets they had never seen. This data pool challenged the arm policy's extrapolation, which was shown to be the most fragile part of all policies, possibly due to error compounding in joints, as the arm features a serial topology. The MLP was the most efficient, with five times fewer parameters, but relies on a hand-designed reference that only works for this expert, whereas the GRU learns that history itself at the cost of size and slower training. So the MLP required more tuning and designing for a higher success rate compared to the GRU which can inherently learn the expert's reference control.

VII. CONCLUSION

In conclusion a Linear Ridge model could not copy the expert in closed loop (0%), while an MLP with a hand-built joint reference and DART data reached 95% on test and a GRU, learning that history itself, reached 95.2% with 98.4% on validation and better robustness to actuator noise than the expert. DART data was found to yield the biggest increase in both networks, showing that to correctly clone the behaviour of the expert noise must be added to the inputs in training. This allowed the networks to perform corrective movements when the robot was moved off track. This error did not affect offline performance, but when tested in closed loop simulation impacted the success rate massively. Furthermore to correctly emulate this specific controller memory was needed so the internal reference could be determined. GRU handled this memory state inherently whereas the MLP policy needed handwritten error handling. This shows that the GRU was best suited for this instance of behaviour cloning. Thus, adding memory, whether designed or learned, together with DART data, mattered far more than lowering offline error.

Limitations and future work. All results are in simulation on flat ground thus, adding obstacles and varying floor friction while training, would show whether the policies transfer to real-hardware performance. Testing on real hardware would also aid the feedback process. Our sensor noise test only changed joint angles and velocities, while the hand-to-target features stayed exact, in the future, sensor noise should also be added to those features. The extra data pool Dagger might have failed due to the expert's reference winding up while the learner drove so integrating the reference reset to the measured joint angles could make it usable. The switch from base to arm control uses the expert's navigation-done flag, so neither policy is fully stand-alone.

REFERENCES

[1] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[2] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise injection for robust imitation learning," in *Proc. Conf. Robot Learning (CoRL)*, 2017, pp. 143–156.

[3] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[4] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[7] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[8] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.

[9] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. [Online]. Available: http://pybullet.org

[10] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979, doi: 10.1080/00401706.1979.10489755.

CONTRIBUTION STATEMENT

- **Noah Calati**: Contributed by coding the training pipeline, ridge regression and data collection with minimal ai usage and scikit-learn tutorials. I then wrote up until MLP section and finalised the conclusion of the paper.
- **Sam Read**: Contributed by building the GRU section of the project. This included the full build, training and testing process. I utilised Noah's training pipeline and also received some assistance from AI tools for some building, interpreting and error checking code.
- **Finn Gorman**: Contributed by building the MLP section of the project. This includes building, iteritvley tuning and testing the MLP along with gathering the data for the models. I built off Noah's original pipeline to make mlp.py and mlp_eval.py with assistance from AI to partially generate code.

All members contributed to writing and reviewing the report.